Artificial Intelligence course

6th Semester

Bachelor in Informatics and Computer Engineering

Nuno Leite

3 March 2023

Introduction to Prolog

- Prolog is one of the classical programming languages developed for applications in AI
  - The other is Lisp (List Processor, 1958)
- Prolog (Programming in logic)
  - Logic-based programming language
  - Programs = sets of logical formulas
  - Prolog interpreter uses logical methods to resolve queries
  - Declarative language
    - Specify what problem you want to solve rather than how to solve it

Your first program: Big and not so big animals

Facts:

```
1 % predicate bigger/2 (2 parameters)
2 bigger(elephant, horse).
3 bigger(horse, donkey).
4 bigger(donkey, dog).
5 bigger(donkey, monkey).
6 % Notice the full stop at the end of each fact
```

Queries

After compilation, we can query the Prolog system:

```
?- bigger(donkey, dog).
true
?- bigger(monkey, elephant).
false
?- bigger(elephant, monkey).
false
% Problem! The last query should succeed!
```

Queries

- We want a predicate that succeeds whenever we can go from the first animal to the second, by iterating the `bigger/2` facts:
  - Transitive closure of `bigger/2`

Rules

```
1 is_bigger(X, Y) :-bigger(X, Y).
2 is_bigger(X, Y) :-bigger(X, Z), is_bigger(Z, Y).
3 % In above, X, Y, Z are variables,
4 % comma (,) is the Logical AND
5 % (:-) means "if"
6 % Note: the order of clauses is important!
```

The above 2 rules define the predicate `is_bigger/2` as the transitive closure of `bigger/2` (via recursion)

Rules

```
?- is_bigger(elephant, monkey).
true
% Query: Are there animals X bigger than a donkey?
?- is_bigger(X, donkey).
X = horse;
X = elephant;
false
% (;) list next alternative
```

Exercise: Illustrate the expansion done in the query:

`?- is_bigger(elephant, monkey).`

Rules

```
?- is_bigger(donkey, X), is_bigger(X, monkey).
false
```

Are there any animals that are both smaller than a donkey and bigger than a monkey?

Terms

- Prolog terms are either:
  - Numbers
  - Atoms
    - Start with a lowercase letter or are enclosed in simple quotes: `elephant`, `xYZ`, `a_123`, `'123'`
  - Variables
    - Start with a capital letter or the underscore: `X`, `Elephant`, `_G177`, `_`
  - Compound terms
    - functor (an *atom*) + number of arguments (*terms*) enclosed in brackets: `is_bigger(horse, X)`, `f(g(X, _), 7)`, `'My Functor'(dog)`
- Atomic terms – atoms and numbers
- Ground terms – terms without variables

Facts and Rules (revisited)

- Facts
  - Compound terms (or atoms), followed by full stop (.)
  - Used to define something as being unconditionally true
    Example: `bigger(elephant, horse).`
- Rules
  - Consist of a head (compound term or atom) and a body (set of compound terms or atoms) separated by `:-`
  - The head of a rule is true if all goals in the body can be proved to be true
    Example:
    `grandfather(X, Y) :- father(X, Z), parent(Z, Y).`

Facts and Rules (revisited)

- Facts and rules are used to define predicates
- Facts and rules are also called clauses
- A program (written in a file) is a list of clauses

Queries (revisited)

- One or several compound terms (or atoms) separated by commas and ending with a full stop (.)
- Typed in the Prolog prompt:

```
?- is_bigger(horse, X), is_bigger(X, dog).
X = donkey
Yes
```

Built-in predicates

Compiling a program file:

```
?- consult('bigger.pl').
Yes
```

Output a term:

```
?- write('Hello World!'), nl.
Hello World!
Yes
```

Matching

- Two terms match if they are identical or can be made identical by substituting variables with ground terms
- We can explicitly ask Prolog whether two terms match using the *equality* predicate =

```
?- born(mary, lisbon) = born(mary, X).
X = lisbon
Yes
```

Matching

```
?- f(a, g(X, Y)) = f(X, Z), Z = g(W, h(X)).
X = a
Y = h(a)
Z = g(a, h(a))
W = a
Yes
```

Matching

```
?- p(X, 2, 2) = p(1, Y, X).
No
```

The anonymous variable (_)

- Every occurrence of _ represents a different variable (which is why instantiations are not being reported in the next example)

```
?- p(_, 2, 2) = p(1, Y, _).
Y = 2
Yes
```

Answering queries

- Means proving that the goal represented by that query can be satisfied (given the program currently in memory)
- Programs are lists of facts (something that is true) and rules (states conditions for something being true)

Algorithm for answering a query

- Algorithm:
  - If a goal matches with a fact, then it is satisfied
  - If a goal matches the head of a rule, then it is satisfied if the goal represented by the rule's body is satisfied
  - If a goal consists of several subgoals separated by commas, then it is satisfied if all its subgoals are satisfied

Algorithm for answering a query

- Note: when trying to satisfy goals using built-in predicates (e.g., `write/1`), Prolog also performs the action associated with it (writing on the screen)
- Example: Mortal Philosophers

  All men are mortal.
  Socrates is a man. (Premises)

  Hence, Socrates is mortal. (Conclusion)

Algorithm for answering a query

- Prolog:

```
1  % Premises
2  mortal(X) :-man(X).
3  man(socrates).
4
5  ?- mortal(socrates).
6  % Conclusion
7  Yes
```

Goal execution

- Algorithm:
  1. `mortal(socrates)` is the initial goal
  2. Prolog tries to match a fact or head of rule with the goal and finds `mortal(X)`; `X = socrates`
  3. This variable instantiation is extended to the rule's body, so `man(X)` $\rightarrow$ `man(socrates)`
  4. New goal: `man(socrates).`
  5. Success, `man(socrates)` is a fact
  6. Therefore, also the initial goal succeeds